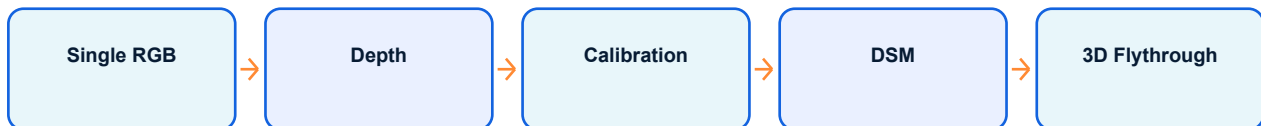


TerraFly Learn-to-Build Roadmap

From zero coding to understanding and rebuilding the complete SIH26175 pipeline



PURPOSE

Follow the order. Each stage contains an easy meaning, required resources, an exact practice task and a proof checkpoint. You do not need to learn all of computer science to understand TerraFly.

Team: TerraFly

Problem Statement: SIH26175 - DepthWizard - Single-View Height Estimation and 3D Flythrough

A practical guide for an honest, measurable, judge-ready system.

1. What this roadmap can realistically do

ONE WEEK

You can become functionally useful: run the app, read the pipeline, explain key choices, change small parts, test outputs and identify fake claims. You will not become independently expert in geospatial AI, 3D graphics and full-stack engineering in seven days.

SIX TO EIGHT WEEKS

With daily practice, you can rebuild the main components from guided examples, debug ordinary problems and defend the implementation with evidence.

TWO TO THREE MONTHS

You can become confident enough to redesign parts, compare models, improve evaluation and teach teammates - if you write code and run experiments instead of only watching videos.

The goal is not to memorize libraries. The goal is to trace one input through every transformation and answer four questions at every step:

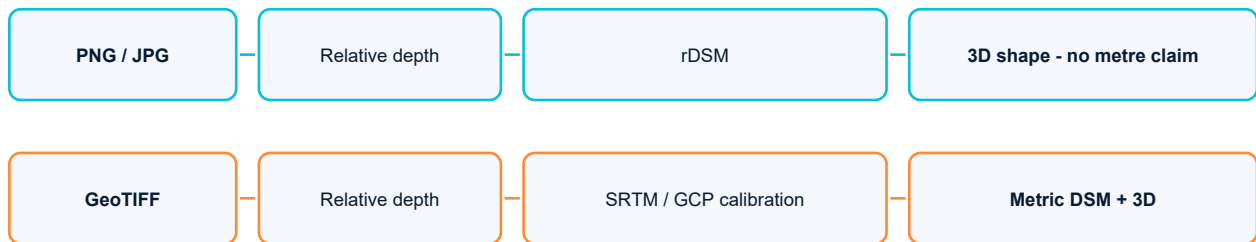
1. What data enters?
2. What operation changes it?
3. What assumptions and units exist?
4. How do we prove the output is reasonable?

Your hardware

Component	What it means for TerraFly
RTX 5060, 8 GB VRAM	Good for inference with selected small/base depth models and moderate tiles. VRAM is the GPU's working memory, not storage. Benchmark before promising image size or speed.
Ryzen 5 7600, 6 cores	Enough for preprocessing, raster operations, mesh generation and CPU fallback at demo scale.
16 GB RAM	Usable but large rasters and meshes can exhaust memory. Downsample/tiling and clean temporary arrays matter.
NVMe + HDD storage	Keep code/models/current samples on NVMe. Keep large archives and backups on the secondary drive/HDD.
Windows	Supported. Use one reproducible Python environment and document exact driver/PyTorch setup.

2. The complete system in plain English

INPUT ROUTING



Stage	Input	Output	Question you must answer
Validate	PNG/JPG/GeoTIFF	Clean image + metadata	What kind of file is this and can metric mode be trusted?
Infer	RGB tensor	Relative depth array	What did the pretrained model predict and in what units?
Calibrate	Relative depth + SRTM/GCP/reference	Metric elevation array	How were scale and offset fitted, aligned and checked?
Evaluate	Prediction + reference	RMSE, MAE, correlation, residuals	Which locations were held out and what fails?
Mesh	DSM/rDSM + RGB	Textured triangles	Are position, height and texture aligned?
Serve	Files and job state	API responses	Can upload, progress, errors and exports be repeated?
View	Mesh + metadata	Interactive flythrough	Can a user navigate and inspect without misunderstanding units?
Package	Code, weights, assets	Local app	Can a second machine run it from the instructions?

THE KEY SCIENTIFIC IDEA

A monocular model usually gives shape/order, not guaranteed metres. Metric DSM is a separate calibrated product. Never blur those two outputs in code, UI, slides or speech.

3. Core glossary - remote sensing and elevation

Official term	Easy meaning
Optical RGB image	A normal colour satellite/aerial image: red, green and blue.
Remote sensing	Observing Earth from a satellite, aircraft or drone without touching the ground.
Single-view	Only one image - no second angle and no stereo pair.
Depth map	The model's estimate of which parts are closer or farther.
Elevation	Height relative to sea level or another defined vertical reference.
DEM	Mostly bare-ground terrain height.
DSM	Height of the visible surface: ground, buildings and trees.
nDSM	Height above local ground: building or tree height, often DSM minus DEM.
rDSM	Relative surface shape; ordering is useful but metres may not be known.
Georeferenced	Every pixel can be mapped to a real location and pixel size.
GeoTIFF	A TIFF that stores coordinates, projection, pixel transform and often NoData information.
SRTM	A widely used near-global elevation dataset, commonly around 30 m sampling.
GCP	A known point whose position and elevation/height are measured.
Scale calibration	Converting relative shape into a metric elevation mapping using reference information.
3D mesh	A surface made of connected triangles raised using elevation values.
Texture	Placing the original RGB image over the 3D surface.
Flythrough	Moving through the generated scene like a game camera or drone.
RMSE	An error score in the output unit that punishes large mistakes strongly.
MAE	The average absolute height/elevation error, normally in metres for metric mode.
Correlation	Whether prediction and reference rise and fall together; it does not prove correct scale.

4. Coding, AI, GIS and 3D glossary

Term	Easy meaning
Python	The main programming language used for image, AI, geospatial and backend work.
Library	Reusable code written by others, such as NumPy, PyTorch or Rasterio.
Array	A grid/list of numbers. An image and a DSM are both arrays.
Tensor	An array used by deep-learning libraries, often placed on GPU memory.
CUDA	NVIDIA's computing platform that lets supported software run suitable calculations on the GPU.
VRAM	Memory on the GPU. Model weights, tensors and intermediate results must fit here.
Inference	Using an already-trained model to make a prediction.
Training	Changing model weights by learning from many labelled examples. Do not assume this is required for the first prototype.
Checkpoint / weights	The saved learned numbers of a trained model.
CRS	The coordinate reference system - rules that say what map coordinates mean.
Affine transform	Six values that map raster row/column positions to geographic coordinates.
NoData	Pixels that are invalid or missing and must not be treated as real height.
Reprojection	Converting data from one CRS/grid to another.
Resampling	Estimating values when changing raster pixel size or grid alignment.
Residual	Reference value minus predicted value after calibration.
Domain gap	The difference between model training images and your overhead satellite/aerial images.
Vertex	One 3D point with x, y and z coordinates.
Triangle	Three connected vertices forming one small piece of the mesh.
UV coordinate	A coordinate that tells the mesh which point of the RGB texture belongs on a vertex.
API	A defined way for the frontend to ask the backend to upload, generate, check status or download.
Frontend / backend	Frontend is the visible app; backend runs files, models, calibration and exports.
Test	An automated or repeatable check that an input produces an expected condition.

5. How to use the links

Label	Meaning
WATCH	Video/course. Watch actively: pause, type the code and predict the result.
READ	Official documentation or paper. Read the listed sections, not every page.
DO	Required hands-on task. You do not pass the stage until it works.
PROVE	Save an artifact or explain it without notes.
OPTIONAL	Useful after the main route; not required for the first functioning app.

The anti-tutorial rule

- For every 30 to 45 minutes of watching, spend at least 30 minutes changing or rebuilding the example.
- Keep one learning repository. Make a commit after each working checkpoint.
- Write a five-line lab note: input, output, units, failure and next step.
- When AI generates code, ask it to explain one function at a time, then deliberately break one input and predict the error.

What you do not need first

- CUDA C/C++ programming, shader mathematics, training a giant model from scratch, Kubernetes, cloud microservices or advanced Blender modelling.
- A three-month general data-science playlist that never touches rasters, calibration or the actual viewer.
- Every React feature. Learn components, state, forms, async requests and routing only as needed.

6. Stage 0 - Windows, folders, Git and reproducibility

EASY MEANING

Before AI, learn how to keep code, environments and changes organized. A reproducible project can be started again after a crash and by another teammate.

Required resources

WATCH/DO: GitHub Hello World - repository, branch, commit and pull request - Beginner-safe and no command line required for the first pass.
<https://docs.github.com/en/get-started/using-github/hello-world>

PRACTICE: GitHub Learn / Skills - Interactive GitHub courses.
<https://skills.github.com/>

READ LATER: Pro Git book - Use chapters 1-3 when you want deeper command-line understanding.
<https://git-scm.com/book/en/v2>

Do

1. Create a TerraFly-learning repository with README, notes, exercises and sample-output folders.
2. Create a Python virtual environment, install one package, save requirements and recreate it once.
3. Make a branch named stage-0, change the README, commit it and merge it.

Prove

Explain repository, commit, branch, environment and requirements without using the words 'some Git thing'. Show the commit history.

7. Stage 1 - Python foundations

EASY MEANING

Python is the glue for TerraFly. You need variables, conditions, loops, functions, files, exceptions, classes at a basic level and package imports. You do not need competitive-programming tricks.

Best beginner course

PRIMARY COURSE: Harvard CS50P - Introduction to Programming with Python - Complete course site with lectures, notes and problem sets.
<https://cs50.harvard.edu/python/>

WATCH FIRST: CS50P Lecture 0 - Functions and Variables - Start here if you have never coded.
<https://www.youtube.com/watch?v=JP7ITIXGpHk>

SHORTS: CS50P Shorts playlist - Short topic explanations for revision.
<https://www.youtube.com/playlist?list=PLhQjrBD2T381eWtNm274cXo8zxUFX-lyn>

REFERENCE: Official Python Tutorial - Use when syntax is unclear; do not read cover to cover first.
<https://docs.python.org/3/tutorial/>

TerraFly-focused syllabus

Learn	TerraFly use
Variables and types	Paths, image sizes, model names, thresholds and job state.
if / for / while	Validate file conditions, process tiles and poll job status.
Functions	Separate load, infer, calibrate, evaluate and export steps.
Lists and dictionaries	Metadata, configuration, results and manifests.
Files and paths	Input, model weights, temporary files and outputs.
Exceptions	Return useful errors instead of crashing.
Classes - basic	Model wrapper or job object; do not over-design.

Do / Prove

Write a program that accepts an image path, rejects unsupported extensions, prints filename and size, creates an output folder and catches a missing-file error. Then explain every line.

8. Stage 2 - NumPy and image arrays

EASY MEANING

An image is a grid of RGB numbers; a depth/DSM is a grid of height-like numbers. NumPy lets you inspect, mask, normalize and transform those grids efficiently.

READ + CODE: NumPy: Absolute Basics for Beginners - Arrays, shape, indexing and basic operations.
https://numpy.org/doc/stable/user/absolute_beginners.html

NEXT: NumPy Quickstart - Broadcasting, reshaping and vectorized operations.
<https://numpy.org/doc/stable/user/quickstart>

WATCH/COURSE: OpenCV official free bootcamp - Image loading, color and common operations.
<https://opencv.org/get-started/>

REFERENCE: OpenCV Python tutorials - Use for reading/writing, resizing, filtering and color conversion.
https://docs.opencv.org/trunk/d6/d00/tutorial_py_root.html

Do

1. Load one RGB image and print its shape, data type, minimum and maximum.
2. Convert it to floating point 0-1, resize it and save a preview.
3. Create a fake 2D height array, mask invalid values, normalize only valid pixels and save a colorized preview.

Prove

Given shape (1024, 2048, 3), explain which dimension is height, width and color channels. Explain why an 8-bit PNG preview is not the same as saving the raw float depth array.

9. Stage 3 - PyTorch, CUDA and inference

CUDA IN ONE SENTENCE

CUDA lets PyTorch use a compatible NVIDIA GPU for tensor operations. It can improve speed; it does not automatically improve accuracy. You do not need to learn CUDA C for TerraFly.

WATCH: PyTorch official beginner YouTube series - Tensors, datasets, model building and training concepts.
<https://docs.pytorch.org/tutorials/beginner/introyt/>

INSTALL: PyTorch Start Locally - Choose Windows, pip and the CUDA option supported by the current PyTorch build.
<https://pytorch.org/get-started/locally/>

WATCH: NVIDIA - What is CUDA? - Conceptual overview; do not turn this into a CUDA-programming detour.
<https://www.youtube.com/watch?v=IzU4AVcMFys>

READ: NVIDIA CUDA FAQ - Understand driver/toolkit/platform terms.
<https://developer.nvidia.com/cuda/faq>

TROUBLESHOOT: CUDA installation guide for Microsoft Windows - Use only if the official PyTorch installation does not work.
<https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/>

Learn

- Tensor shape, batch/channel/height/width, dtype, device, no-grad/inference mode and model eval mode.
- VRAM vs RAM; why larger images create larger intermediate tensors; what out-of-memory means.
- Checkpoint loading, preprocessing and post-processing must match the model repository.

Do / Prove

Create a tensor, move it to GPU when available, run one operation, move it back to CPU and print device/shape/dtype. Then run one pretrained image model in inference mode and measure CPU vs GPU time. Save the exact environment and driver information.

10. Stage 4 - remote sensing and QGIS

EASY MEANING

Before writing geospatial code, use QGIS to see what rasters, bands, CRS, pixel size, terrain and reprojection actually look like.

WATCH: NASA ARSET - Fundamentals of Remote Sensing, Part 1 - Sensors, resolution and basic remote-sensing ideas.
<https://www.youtube.com/watch?v=vGU4UqhHhAA>

WATCH: NASA ARSET - Fundamentals of Remote Sensing, Part 2 - Continue the official foundation.
<https://www.youtube.com/watch?v=rVWfZH2eQY0>

READ: NASA Fundamentals of Remote Sensing PDF - Use as a reference for spatial, spectral, temporal and radiometric resolution.
https://disasters.nasa.gov/sites/default/files/2022-11/Fundamentals_of_RS_Edited_SC.pdf

COURSE: QGIS official training manual - Focus on raster data, raster styling, terrain analysis and reprojection.
https://docs.qgis.org/3.44/en/docs/training_manual/index.html

HINDI PLAYLIST: QGIS for absolute beginners in Hindi - Use if Hindi explanations help you start faster; verify steps against official docs.
https://youtube.com/playlist?list=PLZVI_xLtl0aDdS5RBxr_iwfySOt87OfJP

INDIA: ISRO Bhuvan free webinar/training portal - Indian geospatial platform and learning events.
<https://bhuvan-app1.nrsc.gov.in/bhuvantraining/>

Do / Prove

Open one RGB GeoTIFF and one DEM in QGIS. Show bands, CRS, extent, pixel size and NoData. Reproject one layer, create a hillshade and explain why two rasters can cover the same place but still not have the same grid.

11. Stage 5 - GeoTIFF, CRS, Rasterio, GDAL and PROJ

EASY MEANING

GeoTIFF is not just an image file. Its metadata tells where the pixels belong. Rasterio reads/writes rasters, GDAL provides core geospatial operations and PROJ handles coordinate transformations.

CODE: Rasterio Quickstart - Open a raster and inspect profile, CRS, bounds, transform and bands.
<https://rasterio.readthedocs.io/en/stable/quickstart.html>

CODE: Rasterio reprojection - Put rasters into a common CRS/grid.
<https://rasterio.readthedocs.io/en/stable/topics/reproject.html>

CODE: Rasterio resampling - Understand nearest, bilinear and other choices.
<https://rasterio.readthedocs.io/en/stable/topics/resampling.html>

READ: GDAL geotransform tutorial - Understand pixel-to-map coordinate math.
https://gdal.org/en/stable/tutorials/geotransforms_tut.html

REFERENCE: PROJ usage - Coordinate operations and CRS definitions.
<https://proj.org/en/stable/usage/index.html>

Do

1. Write a raster-inspection tool that prints driver, size, count, dtype, CRS, transform, bounds and NoData.
2. Read a DEM and reproject it exactly onto the RGB GeoTIFF grid. Save the aligned raster.
3. Create a mask showing pixels valid in both prediction and reference.

Prove

Pick pixel row/column, convert it to map coordinates and verify the location in QGIS. Explain why bilinear resampling may suit continuous elevation while nearest-neighbour is safer for class labels.

12. Stage 6 - monocular depth estimation

EASY MEANING

A pretrained monocular model estimates scene geometry from one image. For overhead imagery the prediction can be useful, but the model may have learned from very different ground-level images and usually has no absolute elevation scale.

START HERE: Hugging Face official monocular depth estimation guide - Shows task concepts and an inference workflow.
https://huggingface.co/docs/transformers/tasks/monocular_depth_estimation

MODEL: Depth Anything V2 official repository - Strong open implementation; read model sizes, preprocessing, license and usage.
<https://github.com/DepthAnything/Depth-Anything-V2>

PAPER: Depth Anything V2 paper - Read abstract, method overview, datasets, metrics and limitations.
<https://arxiv.org/abs/2406.09414>

COMPARE: MiDaS official repository - Established relative-depth baseline and model zoo.
<https://github.com/isl-org/MiDaS>

COMPARE: Apple Depth Pro official repository - Another pretrained depth approach; check hardware/license before choosing.
<https://github.com/apple/ml-depth-pro>

What to learn

- Model input size and normalization; output orientation; raw float output; interpolation back to original size.
- Relative vs metric model claims; model license; small/base/large speed and memory trade-offs.
- Domain gap on overhead imagery; shadows, roof shapes, forests, water, seasonal and sensor changes.

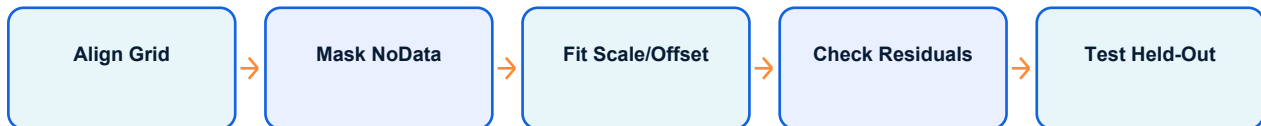
Do / Prove

Run two candidate pretrained models on the same three authorized images. Save raw float arrays, 16-bit/float outputs where appropriate and separate display previews. Record runtime, VRAM, input resize and obvious failure cases. Choose the initial backbone from evidence, not popularity.

13. Stage 7 - calibration: relative shape to metric elevation

EASY MEANING

Calibration learns a mapping such as $\text{elevation} = a \times \text{relative prediction} + b$ using aligned reference pairs. Robust methods reduce the influence of bad points, but they cannot create detail absent from both the prediction and reference.



CODE: scikit-learn RANSAC example - Robust line fitting in the presence of outliers.
https://scikit-learn.org/stable/auto_examples/linear_model/plot_ransac.html

REFERENCE: Rasterio reprojection - Calibration is meaningless if grids are not aligned.
<https://rasterio.readthedocs.io/en/stable/topics/reproject.html>

REFERENCE: Rasterio resampling - Document the interpolation choice.
<https://rasterio.readthedocs.io/en/stable/topics/resampling.html>

Do

1. Create a synthetic relative surface, generate metric reference with known scale/offset and add outliers.
2. Fit ordinary linear regression and RANSAC; compare recovered parameters and residuals.
3. Repeat on aligned real sample data. Separate pixels used for fitting from pixels used for evaluation.

Prove

Show a scatter plot, fitted line, residual histogram/map, number of valid points, inlier threshold and held-out errors. Explain why high correlation can coexist with wrong metre scale.

14. Stage 8 - DSM, slope and scientific evaluation

EASY MEANING

Evaluation compares an aligned prediction with trusted reference values. It is a scientific experiment: the split, units, masks and alignment are as important as the final number.

READ: [scikit-learn model evaluation - Understand error metrics and evaluation structure.](https://scikit-learn.org/stable/modules/model_evaluation.html)
https://scikit-learn.org/stable/modules/model_evaluation.html

CODE: [SciPy Pearson correlation - Use with valid paired pixels and interpret carefully.](https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html)
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html>

TESTING: [pytest getting started - Turn metric and raster assumptions into repeatable tests.](https://docs.pytest.org/en/stable/getting-started.html)
<https://docs.pytest.org/en/stable/getting-started.html>

Required metric card

Field	Example format - replace with real values
Dataset / license	ISPRS Potsdam [terms checked], scene IDs ...
Split	Train/calibration locations ..., held-out test locations ...
Grid	CRS ..., resolution ... m, resampling ..., valid pixels ...
Model	Name, checkpoint, input size, preprocessing, code commit.
Metrics	RMSE [] m, MAE [] m, Pearson r []; per landscape.
Runtime	Image dimensions, device, peak VRAM/RAM and elapsed time.
Failures	Cloud/water/shadow/high-rise/forest examples and warnings.

Do / Prove

Implement metric functions with masks and tests. Create one deliberately misaligned reference and prove the alignment check rejects it. Compute slope from a known plane and verify the expected gradient before using real data.

15. Stage 9 - 3D mesh, texture and flythrough

EASY MEANING

Turn each sampled raster cell into a 3D point, connect neighbouring points into triangles and map the RGB image across them. Large images need downsampling or level-of-detail because every pixel can create too many triangles.

PYTHON: Open3D triangle mesh tutorial - Learn vertices, triangles, normals and mesh operations.
<https://open3d.org/docs/latest/tutorial/Basic/mesh.html>

PYTHON: Open3D NumPy triangle-mesh examples - Build mesh data from arrays.
https://open3d.org/docs/latest/python_example/geometry/triangle_mesh/index.html

EXPORT: Trimesh supported formats - Inspect/export mesh formats such as glTF/GLB.
<https://trimesh.org/formats.html>

WEB: Three.js fundamentals - Scene, camera, renderer, geometry and material.
<https://threejs.org/manual/en/fundamentals.html>

GEOMETRY: Three.js custom BufferGeometry - Create your grid vertices and triangle indices.
<https://threejs.org/manual/en/custom-buffergeometry.html>

TEXTURE: Three.js textures - Map the RGB image correctly.
<https://threejs.org/manual/en/textures.html>

NAVIGATION: Three.js PointerLockControls - First-person mouse navigation.
<https://threejs.org/docs/pages/PointerLockControls.html>

EXPORT: Three.js GLTFExporter - Export interoperable 3D assets.
<https://threejs.org/docs/pages/GLTFExporter.html>

Do / Prove

Build a 64 by 64 synthetic hill mesh first. Confirm triangle count, vertical direction, normals and UV orientation. Then texture it, add orbit and first-person controls, click a point to read x/y/z and export GLB. Only after that use a real DSM/rDSM.

16. Stage 10 - FastAPI backend and job pipeline

EASY MEANING

The backend owns files and heavy processing. The frontend uploads an image, receives a job ID, checks progress and downloads results. Long generation should not freeze one browser request.

COURSE: FastAPI official tutorial - Follow first steps, path operations, request bodies, errors and dependencies as needed.
<https://fastapi.tiangolo.com/tutorial/>

UPLOAD: FastAPI request files - Securely receive PNG/JPG/GeoTIFF inputs.
<https://fastapi.tiangolo.com/tutorial/request-files/>

JOB: FastAPI background tasks - Useful for small local jobs; learn when a real queue would be needed later.
<https://fastapi.tiangolo.com/tutorial/background-tasks/>

FILES: FastAPI static files - Serve local viewer assets/exports when appropriate.
<https://fastapi.tiangolo.com/tutorial/static-files/>

TEST: FastAPI testing - Test upload, status, errors and downloads.
<https://fastapi.tiangolo.com/tutorial/testing/>

Minimum endpoints

Endpoint	Purpose
POST /jobs	Validate upload/configuration and return job ID.
GET /jobs/{id}	Return queued/running/failed/completed, progress and safe error.
GET /jobs/{id}/result	Return manifest and output links only when complete.
GET /health	Report app/model readiness without exposing secrets.

Do / Prove

Implement a fake job that waits through named stages before connecting the model. Test valid upload, bad extension, oversize file, missing job, failed job and completed result. Then replace the fake processing one stage at a time.

17. Stage 11 - JavaScript, React and the judge-facing UI

EASY MEANING

JavaScript runs the interactive browser app. React organizes the interface into components. Vite runs and builds the project. Learn only what is needed for upload, status, results, viewer and errors.

FOUNDATION: MDN Learn Web Development - Use JavaScript fundamentals, forms and async sections.
https://developer.mozilla.org/en-US/docs/Learn_web_development

MODULES: MDN JavaScript modules - Organize viewer, API and UI code.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>

ASYNC: MDN promises - Uploads, job polling and result loading.
https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS/Promises

REACT: React official Learn - Components, props, state, events and conditional rendering.
<https://react.dev/learn>

BUILD: Vite guide - Create and build the frontend.
<https://vite.dev/guide/>

Required screens/states

- Upload/input summary with detected mode; settings; Generate button.
- Progress stages: validate, infer, calibrate, mesh, export.
- Result tabs: input, depth/DSM, residual/metrics, 3D viewer and downloads.
- Permanent unit/mode badge and reference/provenance information.
- Useful empty, loading, success, warning and failure states.

Do / Prove

Build the UI first against the fake FastAPI job. Refresh mid-job, retry a failure and use keyboard navigation. Then connect real outputs. Ask a non-technical teammate to complete the flow without instructions.

18. Stage 12 - testing, packaging and offline operation

EASY MEANING

Packaging means a second machine can launch the complete system without opening your source code and guessing commands. Offline means required weights/assets are local, not that the app is automatically secure.

TEST: pytest getting started - Unit tests for calibration, metrics, masks and file validation.
<https://docs.pytest.org/en/stable/getting-started.html>

PACKAGE: PyInstaller usage - Possible Windows launcher route; test model/data file handling carefully.
<https://pyinstaller.org/en/stable/usage.html>

OPTIONAL: Docker get started - Learn after the local app works; Docker is not mandatory for the first hackathon build.
<https://docs.docker.com/get-started/introduction/>

Test pyramid

Level	Examples
Unit	Metric formula, valid mask, calibration fit, pixel-to-map conversion.
Integration	Upload - inference - export; GeoTIFF - calibration - metrics.
End-to-end	Browser upload - progress - viewer - download on clean machine.
Failure	Bad extension, missing CRS, corrupt raster, out-of-memory, missing model, interrupted job.
Demo	No internet, projector resolution, prepared tile, recording fallback.

Do / Prove

Create one launch command/button, a health check and a clean-machine checklist. Test with network disconnected after all authorized dependencies are local. Record startup and processing time, peak memory and any setup still requiring internet.

19. Data sources - use, but check every license

DEM: NASA SRTM Quick Guide - Understand coverage, products and limitations.
https://www.earthdata.nasa.gov/s3fs-public/2025-05/SRTM_Quick_Guide.pdf

DOWNLOAD: NASA Earthdata Search - Find SRTM and other authorized Earth-observation data; account may be required.
<https://search.earthdata.nasa.gov/search>

INDIA: Bhuvan data download help - Indian portal and product-download guidance, including available elevation products.
<https://bhuvan-app3.nrsc.gov.in/data/download/help/source/html/introduction.htm>

DEM: Copernicus DEM collection - Alternative DEM reference; check access and product terms.
<https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM>

RGB + DSM: ISPRS 2D semantic labelling benchmarks - Potsdam and Vaihingen - Very high resolution orthophotos and DSMs; check challenge/license conditions.
<https://isprs.org/resources/datasets/benchmarks/UrbanSemLab/semantic-labeling.aspx>

RGB + LIDAR/DSM: IEEE GRSS Data Fusion Contest 2018 - Useful paired data but with contest/publication and redistribution conditions.
<https://www.grss-ieee.org/community/technical-committees/2018-ieee-grss-data-fusion-contest/>

LIDAR/DEM: OpenTopography - getting started - Find high-resolution topographic data; individual datasets have their own terms.
<https://www.opentopography.org/start>

WATCH: OpenTopography tutorial video - How to discover and access data.
<https://www.youtube.com/watch?v=AouYvcVlxvc>

License ledger fields

Field	Record
Source and product	Exact page, product name, version and acquisition date.
License / terms	Training allowed? Demo allowed? Redistribution allowed? Attribution required?
Spatial/sensor details	Location, date, resolution, bands, CRS and vertical reference.
Use in project	Calibration, evaluation, screenshot, sample or training.

DO NOT SCRAPE GOOGLE EARTH

Do not train on or redistribute scraped Google Earth imagery. Use open/licensed data and authorized inputs.
 Record provenance for every sample shown to judges.

20. Mini-project ladder - proof that you learned

#	Mini-project	Pass condition
1	Image inspector	Validates path/type; prints RGB shape/dtype/size; handles missing file.
2	Array lab	Normalizes/masks a fake height grid and preserves raw float values.
3	CUDA inference check	Prints CPU/GPU device and times one pretrained inference.
4	GeoTIFF inspector	Prints CRS/bounds/transform/NoData and verifies one pixel coordinate.
5	Raster aligner	Reprojects a DEM exactly onto an RGB/reference grid and saves valid mask.
6	Depth comparer	Runs two backbones and records raw output, runtime, VRAM and failures.
7	Calibration lab	Fits scale/offset with held-out evaluation and residual plot.
8	Metric card	Computes masked RMSE, MAE and Pearson correlation with tests.
9	Textured hill	Creates 64x64 grid mesh, correct UV texture, controls and GLB export.
10	Fake job app	Upload - job ID - progress - result - failure states across frontend/backend.
11	Integrated relative route	PNG/JPG - raw relative depth - rDSM preview - textured 3D - export.
12	Integrated metric route	GeoTIFF - aligned reference - calibrated DSM - metrics - 3D - GeoTIFF export.
13	Clean-machine launch	Teammate follows README on second machine/profile without your help.

RULE

Do not mark a stage complete because the video ended. Mark it complete because the mini-project works, you can explain it and the artifact is saved in Git.

21. Seven-day crash route - become useful fast

Day	Learn	Build / evidence
1	Python essentials, paths, exceptions, Git; NumPy image shape and dtype.	Mini-projects 1 and 2. Explain RGB array and raw vs preview.
2	PyTorch tensors, inference mode, CUDA/VRAM; run selected depth backbone.	Mini-projects 3 and 6 on small authorized samples.
3	QGIS, raster metadata, CRS, transform, NoData, reprojection/resampling.	Mini-projects 4 and 5; verify alignment in QGIS.
4	Calibration, masks, RMSE/MAE/correlation, held-out split.	Mini-projects 7 and 8 with synthetic then real sample.
5	Mesh grid, triangles, UV, texture, Three.js camera/controls.	Mini-project 9; connect one actual rDSM.
6	FastAPI upload/jobs and React async UI; connect relative route.	Mini-projects 10 and 11.
7	Metric route, tests, packaging, failure rehearsal and explanation.	Mini-projects 12 and 13; record demo and metric card.

TIME MANAGEMENT

During the crash route, use the course links selectively. Spend more time on the Do/Prove task than on passive watching. If the metric route is not scientifically valid by Day 7, demo relative mode honestly and show the calibration experiment separately.

22. Eight-week mastery route

Week	Focus	End-of-week capability
1	CS50P essentials + Git	Write and debug small file-processing programs with tests.
2	NumPy, OpenCV, plotting	Manipulate images/height arrays and preserve numerical data.
3	PyTorch, CUDA, pretrained inference	Integrate and benchmark a model without blindly copying code.
4	Remote sensing, QGIS, CRS, Rasterio	Inspect, align and export geospatial rasters correctly.
5	Depth domain gap, calibration and evaluation	Run controlled experiments and defend metrics/limitations.
6	Mesh math, Three.js, interaction	Build a textured terrain viewer and explain geometry/units.
7	FastAPI, React and job states	Build the integrated upload-to-result application.
8	Testing, packaging, profiling and communication	Rebuild on a clean machine, publish proof package and teach teammates.

After week 8

- Compare backbones on a fixed benchmark and document speed/quality trade-offs.
- Try robust/local calibration alternatives and landscape-specific analysis.
- Add tiling/overlap handling and level-of-detail after measuring seams and memory.
- Read remote-sensing monocular-depth research papers relevant to overhead imagery instead of assuming natural-image results transfer.
- Write a technical report with repeatable commands, data cards and ablation studies.

23. How to understand AI-generated code

Use this five-pass method for every important file

1. Map: write one sentence for the file, every function and every input/output.
2. Trace: follow one sample from upload through array/tensor/raster/mesh. Write shapes, dtypes, CRS and units at boundaries.
3. Predict: before running, state what should happen for one valid input and two failures.
4. Break: remove metadata, change shape, pass NoData, disable GPU or interrupt a job. Confirm the failure is safe and understandable.
5. Rebuild: rewrite one small function from tests and documentation without looking at the generated version.

Questions to ask AI

- Explain this function to a beginner using the exact input/output shapes, dtype, CRS and units.
- Which assumptions could silently make this output scientifically wrong?
- Write tests for a valid case, missing metadata, all-NoData input and a deliberately misaligned raster.
- Show the smallest version of this function using official library APIs, then explain every line.
- What source/license does this model or data require? Do not guess; link the official source.

You own the code only when

Check	Pass
Explain	You can explain every major transformation without reading generated comments.
Modify	You can change one setting/algorithm and predict the effect.
Test	You can show success and failure tests.
Measure	You know runtime, memory and metric evidence.
Limit	You can name what the code cannot guarantee.

24. Final self-test before saying 'I understand TerraFly'

1. Why can one RGB image provide relative shape but not generally absolute elevation?
2. What exact metadata makes a GeoTIFF georeferenced?
3. Why must the reference DEM and prediction share CRS, grid, resolution and valid mask?
4. What does CUDA do, and what does it not do?
5. How are RMSE, MAE and correlation different?
6. Why can SRTM anchor broad scale but not prove fine rooftop detail?
7. How do pixels become vertices, triangles and texture coordinates?
8. What happens from POST /jobs until the viewer loads?
9. Which output files are numerical truth and which are display previews?
10. What data/model licenses apply?
11. What are the three biggest failure conditions and how does the UI warn the user?
12. Can a second machine launch the app from the README?

PASS STANDARD

Answer ten of twelve clearly, demonstrate the relevant artifact for at least eight, and be honest about the two you cannot yet answer. Then teach the same map to a teammate.

25. Resource index

Python: Harvard CS50P

<https://cs50.harvard.edu/python/>

Python: Official Python Tutorial

<https://docs.python.org/3/tutorial/>

Arrays: NumPy beginner guide

https://numpy.org/doc/stable/user/absolute_beginners.html

Vision: OpenCV Python tutorials

https://docs.opencv.org/trunk/d6/d00/tutorial_py_root.html

AI: PyTorch official beginner series

<https://docs.pytorch.org/tutorials/beginner/introyt/>

GPU: PyTorch install selector

<https://pytorch.org/get-started/locally/>

Remote sensing: NASA ARSET

<https://appliedsciences.nasa.gov/what-we-do/capacity-building/arset/about-arset>

GIS: QGIS training manual

https://docs.qgis.org/3.44/en/docs/training_manual/index.html

GIS India: QGIS India learning resources

<https://in.qgis.org/learning-resources/>

Raster: Rasterio quickstart

<https://rasterio.readthedocs.io/en/stable/quickstart.html>

This compact index lists the foundation links. The depth-model, calibration, 3D, backend, frontend, testing, packaging and dataset links are placed directly inside their relevant stages. Courses and repositories can change, so save version/commit information and prefer official documentation and original repositories over copied tutorials.